lmbench3: measuring scalability

Carl Staelin
HP Laboratories Israel
HPL-2002-313
November 8th , 2002*

lmbench, benchmarking, distributed systems, parallel systems, multi-processor systems

`lmbench3` extends the `lmbench2` system to measure a system's performance under scalable load to make it possible to assess parallel and distributed computer performance with the same power and flexibility that `lmbench2` brought to uni-processor performance analysis. There is a new timing harness, `benchmp`, designed to measure performance at specific levels of parallel (simultaneous) load, and most existing benchmarks have been converted to use the new harness.

`lmbench` is a micro-benchmark suite designed to focus attention on the basic building blocks of many common system applications, such as databases, simulations, software development, and networking. It is also designed to make it easy for users to create additional micro-benchmarks that can measure features, algorithms, or subsystems of particular interest to the user.

* Internal Accession Date Only
Approved for External Publication
Technion City, Haifa 32000, Israel

lmbench3: measuring scalability

Carl Staelin
Hewlett-Packard Laboratories Israel

ABSTRACT

lmbench3 extends the lmbench2 system to measure a system's performance under scalable load to make it possible to assess parallel and distributed computer performance with the same power and flexibility that lmbench2 brought to uni-processor performance analysis. There is a new timing harness, benchmp, designed to measure performance at specific levels of parallel (simultaneous) load, and most existing benchmarks have been converted to use the new harness.

lmbench is a micro-benchmark suite designed to focus attention on the basic building blocks of many common system applications, such as databases, simulations, software development, and networking. It is also designed to make it easy for users to create additional micro-benchmarks that can measure features, algorithms, or subsystems of particular interest to the user.

1. Introduction

lmbench is a widely used suite of micro-benchmarks that measures important aspects of computer system performance, such as memory latency and bandwidth. Crucially, the suite is written in portable ANSI-C using POSIX interfaces and is intended to run on a wide range of systems without modification.

The benchmarks included in the suite were chosen because in the lmbench developer's experience, they each represent an aspect of system performance which has been crucial to an application's performance. Using this multi-dimensional performance analysis approach, it is possible to better predict and understand application performance because key aspects of application performance can often be understood as linear combinations of the elements measured by lmbench[4].

lmbench3 extends the lmbench suite to encompass parallel and distributed system performance by measuring system performance under scalable load. This means that the user can specify the number of processes that will be executing the benchmarked feature in parallel during the measurements. It is possible to utilize this framework to develop benchmarks to measure distributed application performance, but it is primarily intended to measure the performance of multiple processes using the same system resource at the same time.

In general the benchmarks report either the latency or bandwidth of an operation or data pathway. The exceptions are generally those benchmarks that report on a specific aspect of the hardware, such as the processor clock rate, which is reported in MHz and nanoseconds.

lmbench consists of three major components: a timing harness, the individual benchmarks built on top of the timing harness, and the various scripts and glue that build and run the benchmarks and process the results.

1.1. lmbench history

lmbench1 was written by Larry McVoy while he was at Sun Microsystems. It focussed on two measures of system performance: latency and bandwidth. It measured a number of basic operating system functions, such as file system read/write bandwidth or file creation time. It also focussed a great deal of energy on measuring data transfer operations, such as bcopy and pipe latency and bandwidth as well as raw memory latency and bandwidth.

Shortly after the lmbench1 paper [12] was published, Aaron Brown examined the lmbench benchmark suite and published a detailed critique of its strengths and weaknesses[4]. Largely in response to these remarks, development of lmbench2 began with a focus on improving the experimental design and statistical data analysis. The primary change was the development and adoption across all the benchmarks of a timing harness that incorporated loop-autosizing and clock resolution detection. In addition, each experiment was typically repeated eleven times with the median result reported to the user.

The lmbench2[21] timing harness was implemented through a new macro, BENCH(), that automatically manages nearly all aspects of accurately timing operations. For example, it automatically detects the minimal timing interval necessary to provide timing results within 1% accuracy, and it automatically repeats most experiments eleven times and reports the median

result.

lmbench3 focussed on extending lmbench's functionality along two dimensions: measuring multi-processor scalability and measuring basic aspects of processor micro-architecture.

An important feature of multi-processor systems is their ability to scale their performance. While lmbench1 and lmbench2 measure various important aspects of system performance, they cannot measure performance with more than one client process active at a time. Consequently, measuring performance of multi-processor and clustered systems as a function of scalable load was impossible using those tools.

lmbench3 took the ideas and techniques developed in the earlier versions and extended them to create a new timing harness which can measure system performance under parallel, scalable loads.

lmbench3 also includes a version of John McCalpin's STREAM benchmarks. Essentially the STREAM kernels were placed in the new lmbench timing harness. Since the new timing harness also measures scalability under parallel load, the lmbench3 STREAM benchmarks include this capability automatically.

Finally, lmbench3 includes a number of new benchmarks which measure various aspects of the processor architecture, such as basic operation latency and parallelism, to provide developers with a better understanding of system capabilities. The hope is that better informed developers will be able to better design and evaluate performance critical software in light of their increased understanding of basic system performance.

2. Prior Work

Benchmarking is not a new field of endeavor. There are a wide variety of approaches to benchmarking, many of which differ greatly from that taken by lmbench.

One common form of benchmark is to take an important application or application and worklist, and to measure the time required to complete the entire task. This approach is particularly useful when evaluating the utility of systems for a single and well-known task.

Other benchmarks, such as SPECint, use a variation on this approach by measuring several applications and combining the results to predict overall performance. SPEChpc96 [22] extends this approach to the parallel and distributed domain by measuring the performance of a selected parallel applications built on top of MPI and/or PVM.

Another variation takes the "kernel" of an important application and measures its performance, where the "kernel" is usually a simplification of the most expensive portion of a program. Dhrystone [23] is an example of this type of benchmark as it measures the performance of important matrix operations and was often used to predict system performance for numerical operations.

Banga developed a benchmark to measure HTTP server performance which can accurately measure server performance under high load[1]. Due to the idiosyncracies of the HTTP protocol and TCP design and implementation, there are generally operating system limits on the rate at which a single system can generate independent HTTP requests. However, Banga developed a system which can scalably present load to HTTP servers in spite of this limitation[2].

John McCalpin's STREAM benchmark measures memory bandwidth during four common vector operations[11]. It does not measure memory latency, and strictly speaking it does not measure raw memory bandwith although memory bandwidth is crucial to STREAM performance. More recently, STREAM has been extended to measure distributed application performance using MPI to measure scalable memory subsystem performance, particularly for multi-processor machines.

Prestor[16] and Saavedra[17] have developed benchmarks which analyze memory subsystem performance.

Micro-benchmarking extends the "kernel" approach, by measuring the performance of operations or resources in isolation. lmbench and many other benchmarks, such as nfsstone[20], measure the performance of key operations so users can predict performance for certain workloads and applications by combining the performance of these operations in the right mixture.

Saavedra[18] takes the micro-benchmark approach and applies it to the problem of predicting application performance. They analyze applications or other benchmarks in terms of their ‘‘narrow spectrum benchmarks’’ to create a linear model of the application's computing requirements. They then measure the computer system's performance across this set of micro-benchmarks and use a linear model to predict the application's performance on the computer system. Seltzer[19] applied this technique using the features measured by lmbench as the basis for application prediction.

Benchmarking I/O systems has proven particularly troublesome over the years, largely due to the strong non-linearities exhibited by disk systems. Sequential I/O provides much higher bandwidth than non-sequential I/O, so performance is highly dependent on the workload characteristics as well as the file system's ability to capitalize on available sequentiality by laying out data contiguously on disk.

I/O benchmarks have a tendency to age poorly. For example, IOStone[15], IOBench[24], and the Andrew benchmark[8] used fixed size datasets, whose size was significant at the time, but which no longer measure I/O performance as the data can now fit in the

processor cache of many modern machines.

The Andrew benchmark attempts to separately measure the time to create, write, re-read, and then delete a large number of files in a hierarchical file system.

Bonnie[3] measures sequential, streaming I/O bandwidth for a single process, and random I/O latency for multiple processes.

Peter Chen developed an adaptive harness for I/O benchmarking[6][5], which defines I/O load in terms of five parameters, uniqueBytes, sizeMean, readFrac, seqFrac, and processNum. The benchmark then explores the parameter space to measure file system performance in a scalable fashion.

Parkbench[14] is a benchmark suite that can analyze parallel and distributed computer performance. It contains a variety of benchmarks that measure both aspects of system performance, such as communication overheads, and distributed application kernel performance. Parkbench contains benchmarks from both NAS[13] and Genesis[7].

3. Timing Harness

The first, and most crucial element in extending `lmbench2` so that it could measure scalable performance, was to develop a new timing harness that could accurately measure performance for any given load. Once this was done, then each benchmark would be migrated to the new timing harness.

The harness is designed to accomplish a number of goals:

1. during any timing interval of any child it is guaranteed that all other child processes are also running the benchmark
2. the timing intervals are long enough to average out most transient OS scheduler affects
3. the timing intervals are long enough to ensure that error due to clock resolution is negligible
4. timing measurements can be postponed to allow the OS scheduler to settle and adjust to the load
5. the reported results should be representative and the data analysis should be robust
6. timing intervals should be as short as possible while ensuring accurate results

Developing an accurate timing harness with a valid experimental design is more difficult than is generally supposed. Many programs incorporate elementary timing harnesses which may suffer from one or more defects, such as insufficient care taken to ensure that the benchmarked operation is run long enough to ensure that the error introduced by the clock resolution is insignificant. The basic elements of a good timing harness are discussed in Staelin[21].

The new timing harness must also collect and process the timing results from all the child processes so that it can report the representative performance. It currently reports the median performance over all timing intervals from all child processes. It might perhaps be argued that it should report the median of the medians.

When running benchmarks with more than one child, the harness must first get a baseline estimate of performance by running the benchmark in only one process using the standard `lmbench` timing interval, which is often 5,000 microseconds. Using this information, the harness can compute the average time per iteration for a single process, and it uses this figure to compute the number of iterations necessary to ensure that each child runs for at least one second.

3.1. Clock resolution

`lmbench` uses the `gettimeofday` clock, whose interface resolves time down to 1 microsecond. However, many system clock's resolution is only 10 milliseconds, and there is no portable way to query the system to discover the true clock resolution.

The problem is that the timing intervals must be substantially larger than the clock resolution in order to ensure that the timing error doesn't impact the results. For example, the true duration of an event measured with a 10 milli-second clock can vary ±10 milli-seconds from the true time, assuming that the reported time is always a truncated version of the true time. If the clock itself is not updated precisely, the true error can be even larger. This implies that timing intervals on these systems should be at least 1 second.

However, the `gettimeofday` clock resolution in most modern systems is 1 microsecond, so timing intervals can as small as a few milli-seconds without incurring significant timing errors related to clock resolution.

Since there is no standard interface to query the operating system for the clock resolution, `lmbench` must experimentally determine the appropriate timing interval duration which provides results in a timely fashion with a negligible clock resolution error.

3.2. Coordination

Developing a timing harness that correctly manages *N* processes and accurately measures system performance over those same *N* processes is significantly more difficult than simply measuring system performance with a single process because of the asynchronous nature of parallel programming.

In essence, the new timing harness needs to create *N* jobs, and measure the average performance of the target subsystem while all *N* jobs are running. This is a standard problem for parallel and distributed programming, and involves starting the child processes and then stepping through a handshaking process to ensure that all children have started executing the benchmarked operation before any child starts taking measurements.

Parent	Child
• start up P child processes	
• wait for P *ready* signals	• run benchmark operation for a little while
↓	• send a *ready* signal
• on reciept of *ready* signals, sleep for *warmup* `micros`	• run benchmark operation while polling for a *go* signal
• send *go* signal to P children	↓
• wait for P *done* signals	• on receipt of *go* signal, begin timing benchmark operation
↓	• send a *done* signal
• one receipt of *done* signals, iterate through children sending *results* signal and gathering results	• run benchmark operation while polling for a *results* signal
• collate results	• on receipt of *results* signal, send timing results and wait for *exit* signal
• send *exit* signal	↓
	• exit

Table 1. Timing harness sequencing

Table 1 shows how the parent and child processes coordinate their activities to ensure that all children are actively running the benchmark activity while any child could be taking timing measurements.

The reason for the separate "exit" signal is to ensure that all properly managed children are alive until the parent allows them to die. This means that any SIGCHLD events that occur before the "exit" signal indicate a child failure.

3.3. Accuracy

The new timing harness also needs to ensure that the timing intervals are long enough for the results to be representative. The previous timing harness assumed that only single process results were important, and it was able to use timing intervals as short as possible while ensuring that errors introduced by the clock resolution were negligible. In many instances this meant that the timing intervals were smaller than a single scheduler time slice. The new timing harness must run benchmarked operations long enough to ensure that timing intervals are longer than a single scheduler time slice. Otherwise, you can get results which are complete nonsense. For example, running several copies of an `lmbench2` benchmark on a uni-processor machine will often report that the per-process performance with *N* jobs running in parallel is equivalent to the performance with a single job running![1]

In addition, since the timing intervals now have to be longer than a single scheduler time slice, they also need to be long enough so that a single scheduler time slice is insignificant compared to the timing interval. Otherwise the timing results can be dramatically affected by small variations in the scheduler's behavior.

Currently `lmbench` does not measure the scheduler timeslice; the design blithely assumes that timeslices are generally on the order of 10-20ms, so one second timing intervals are sufficient. Some schedulers may utilize longer time slices, but this has not (yet) been a problem.

3.4. Resource consumption

One important design goal was that resource consumption be constant with respect to the number of child processes. This is why the harness uses shared pipes to communicate with the children, rather than having a separate set of pipes to communicate with each child. An early design of the system utilized a pair of pipes per child for communication and synchronization between the master and slave processes. However, as the number of child processes grew, the fraction of system resources consumed by the harness grew and the additional system overhead could start to interfere with the accuracy of the measurements.

Additionally, if the master has to poll (`select`) *N* pipes, then the system overhead of that operation also scales with the number of children.

3.5. Pipe atomicity

Since all communication between the master process and the slave (child) processes is done via a set of shared pipes, we have to ensure that we never have a situation where the message can be garbled by the intermingling of two separate messages from two separate children. This is ensured by either using pipe operations that are guaranteed to be atomic on all machines, or by coordinating between processes so that at most one process is writing at a time.

The atomicity guarantees are provided by having each client communicate synchronization states in one-byte messages. For example, the signals from the master to each child are one-byte messages, so each child only reads a single byte from the pipe. Similarly, the responses from the children back to the master are also one-byte messages. In this way no child can receive partial messages, and no message can be interleaved with any other message.

However, using this design means that we need to have a separate pipe for each *barrier* in the process, so

[1] This was discovered by someone who naively attempted to parallelize `lmbench2` in this fashion, and I received a note from the dismayed developer describing the failed experiment.